

Lab-three: KVDataBase (Raft)

实验记录

实验要求

1. 核心架构：分层设计

- **KVServer 层 (Upper Layer)**: 负责处理客户端的 `Put`、`Append`、`Get` 请求，维护内存中的哈希表。
- **Raft 层 (Lower Layer)**: 负责日志复制和达成共识。

工作流简述：

1. **Client** 向 **KVServer** 发送 RPC 请求。
2. **KVServer** 调用 `rf.Start(command)` 将操作提交给 Raft。
3. **Raft** 在集群间同步该日志，并在达成多数派共识后，通过 `applyCh` 将该指令传回 **KVServer**。
4. **KVServer** 执行指令并返回结果给 **Client**。

2. 关键框架代码含义

`client.go` (Clerk)

- `Put/Append/Get`: 客户端调用的接口。由于客户端不知道谁是当前的 Leader，它会轮询服务器，直到找到 Leader 为止。
- `ClientId` & `SeqNum`: 这是实现 **幂等性 (Idempotency)** 的关键。每个客户端分配一个唯一 ID，每个请求分配一个递增序列号，防止重复执行。

`server.go` (KVServer)

- `kv.kvStore`: 实际存储数据的 `map[string]string`。
- `kv.waitCh`: 一个 `map[int]chan Op`。当 `Start()` 返回后，Server 需要在一个 channel 上阻塞等待，直到 Raft 异步地将这条日志通过 `applyCh` 传回。
- `kv.lastApplied`: 记录每个客户端最后执行成功的 `SeqNum`，用于去重。

`common.go`

- 定义了 RPC 的参数和返回值 (`PutAppendArgs`, `GetReply` 等)。

3. 实现中的核心难点

A. 读操作也必须走 Raft

为了保证**线性一致性 (Linearizability)**，`Get` 请求不能直接读取本地内存。虽然 `Get` 不改变数据，但它必须确认当前服务器仍然是 Leader。

提示：最简单的做法是将 `Get` 也打包成一条 Raft 日志，像 `Put` 一样走一遍流程。

B. 处理重复请求 (At-most-once Semantics)

网络不稳定会导致客户端重试。如果不加处理，`Append` 操作可能会执行多次。

- **方案：**在 Server 端维护一个 `map[ClientId]LastSeqNum`。在应用日志到状态机之前，先检查当前请求的 `SeqNum` 是否已经执行过。

C. 处理日志丢失与 Leader 切换

当 `rf.Start()` 成功后，并不代表日志一定会被提交。如果中途发生了 Leader 切换，该日志可能会被覆盖。

- **解决方案：**KVServer 监听 `applyCh` 时，需要检查返回的日志内容是否和当初 `Start()` 发送的一致。如果不一致，说明旧 Leader 已经失权，应向客户端返回错误，让其重试。

实现细节

实验三中涉及到三个代码文件 `client.go` / `common.go` / `server.go` 整个系统一同协作保证幂等性

`common.go` 定义RPC请求用到的结构体和响应消息

`client.go` 定义客户端用于分配clientId和发送请求（为每个操作生成全局唯一的 **(ClientId, SeqId)**）

`server.go` 作为服务层配合RAFT完成客户端发来的请求

客户端：

1. 客户端ID生成 64位

[63:32] Unix 秒时间戳（32bit）

[31:10] 随机数（22bit，每秒重采样）

[9:0] 同秒内计数器（10bit，0~1023 回绕）

2. 操作ID增加

`SeqId` 通过 `atomic.AddInt64` 单调递增，确保每个请求有唯一序号

3. 请求发送与重试

OK / ErrNoKey 成功返回

ErrWrongLeader 轮转到下一台 server

ErrTimeout 保持相同 SeqId 继续重试当前 server

ok=false 轮转到下一台 server

服务端：

1. 接受请求并发送到raft层中

注册到RPC中的两个handler函数将客户端发来的请求数据封装为Op结构体

传给 `submitAndWait()` 函数：调用raft层的start函数，得到对应请求索引和该服务器是否为leader

判断请求索引是否已经存在，已存在就对旧请求返回错误leader，让旧请求重试，并用新请求覆盖

然后得到chan得到通知，然后返回给client，返回前会删除自己创建的通知通道

2. applier协程将raft提交的日志应用到状态机

开启applier协程接收raft发送的提交日志

如果是快照，就先将快照安装到raft层，再将快照应用到状态机

如果是日志，查询去重表看看请求是否已经过期

过期了，直接返回最新的结果

没过期，调用applyOp函数直接应用日志请求到状态机

判断日志长度，太大了就生成快照

然后通过chan返回结果

3. 快照

快照分为两部分加载快照和生成快照

加载快照：如果节点发生了意外，进行重启，用于快速恢复状态

生成快照：如果日志数据超过了规定的最大长度，就生成快照压缩日志

遇到的BUG

Bug1: index 被不同请求覆盖

场景：leader 在某 index 提交后崩溃重选，新 leader 在同一 index 提交了另一个请求，旧的等待 goroutine 会收到错误结果。

修复： `notifyEntry` 存储 `clientId+seqId`，apply 时双重校验，不匹配则不通知；同时新请求注册时若发现 index 被占用，先向旧 channel 推送 `ErrWrongLeader`。

Bug2: 持锁发送 channel 导致死锁

场景：applier 持锁时向 channel 发送，若 channel 满且接收方也在等锁，双方互相阻塞。

修复：在锁内仅取出 channel 引用，**释放锁后**再执行发送。channel 容量设为 1，保证发送不阻塞。

Bug3：超时后轮转导致重复执行

场景：操作已在 Raft 中 apply 但 RPC 超时，client 轮转到其他 server 用相同 SeqId 重试，若未命中去重表则被重复执行。

修复：超时时保持在当前 server 重试，由 `dupTable` 保证幂等性；只有 RPC 彻底失败（网络断开）时才轮转。

幂等性保障

幂等性通过客户端唯一标识 + 序列号与服务端去重表协同保证。核心目标：同一个 `(ClientId, SeqId)` 的请求无论因超时、重试、网络延迟等原因被提交多少次，状态机只执行一次，并返回相同的响应。

common.go

```
1  package kvraft
2
3  const (
4      OK                = "OK"
5      ErrNoKey          = "ErrNoKey"
6      ErrWrongLeader    = "ErrWrongLeader"
7      ErrTimeout        = "ErrTimeout"
8  )
9
10 type Err string
11
12 // ===== PutAppend =====
13
14 type PutAppendArgs struct {
15     Key    string
16     Value  string
17     Op     string // "Put" or "Append"
18     ClientId int64 // 客户端唯一 ID (由 nrand() 生成)
19     SeqId   int64 // 单调递增的请求序列号, server 用它去重
20 }
21
22 type PutAppendReply struct {
23     Err Err
24 }
25
```

```

26 // ===== Get =====
27
28 type GetArgs struct {
29     Key string
30     ClientId int64
31     SeqId int64
32 }
33
34 type GetReply struct {
35     Err Err
36     Value string
37 }

```

server.go

```

1 package kvraft
2
3 import (
4     "6.824/labgob"
5     "6.824/labrpc"
6     "6.824/raft"
7     "bytes"
8     "log"
9     "sync"
10    "sync/atomic"
11    "time"
12 )
13
14 const Debug = false
15
16 func DPrintf(format string, a ...interface{}) (n int, err error) {
17     if Debug {
18         log.Printf(format, a...)
19     }
20     return
21 }
22
23 const ApplyTimeout = 500 * time.Millisecond
24
25 // ===== Op =====
26
27 type Op struct {
28     OpType string
29     Key string
30     Value string
31     ClientId int64

```

```

32     SeqId    int64
33 }
34
35 // ===== applyResult =====
36
37 type applyResult struct {
38     Err      Err
39     Value    string
40     ClientId int64
41     SeqId    int64
42 }
43
44 // ===== 去重表条目 =====
45
46 type lastReply struct {
47     SeqId int64
48     Reply applyResult
49 }
50
51 // ===== notifyEntry: 带身份标识的等待项 =====
52 //
53 // Bug1 修复: 用 clientId+seqId 标识 channel 归属。
54 // 新请求注册前检查 index 是否已被占用 (极少发生),
55 // apply 时只通知 clientId+seqId 匹配的 channel,
56 // 避免 leader 切换后同一 index 被不同请求覆盖。
57 //
58
59 type notifyEntry struct {
60     clientId int64
61     seqId    int64
62     ch       chan applyResult
63 }
64
65 // ===== KVServer =====
66
67 type KVServer struct {
68     mu      sync.Mutex
69     me      int
70     rf      *raft.Raft
71     applyCh chan raft.ApplyMsg
72     dead    int32
73
74     maxraftstate int
75     persister    *raft.Persister
76
77     kvStore    map[string]string
78     dupTable   map[int64]lastReply

```

```

79     notifyChans map[int]notifyEntry // index -> 等待项
80 }
81
82 // ===== RPC Handler: Get =====
83
84 func (kv *KVServer) Get(args *GetArgs, reply *GetReply) {
85     op := Op{
86         OpType:  "Get",
87         Key:      args.Key,
88         ClientId: args.ClientId,
89         SeqId:    args.SeqId,
90     }
91     err, value := kv.submitAndWait(op)
92     reply.Err = err
93     reply.Value = value
94 }
95
96 // ===== RPC Handler: PutAppend =====
97
98 func (kv *KVServer) PutAppend(args *PutAppendArgs, reply *PutAppendReply) {
99     op := Op{
100         OpType:  args.Op,
101         Key:      args.Key,
102         Value:    args.Value,
103         ClientId: args.ClientId,
104         SeqId:    args.SeqId,
105     }
106     err, _ := kv.submitAndWait(op)
107     reply.Err = err
108 }
109
110 // ===== submitAndWait =====
111
112 func (kv *KVServer) submitAndWait(op Op) (Err, string) {
113     index, _, isLeader := kv.rf.Start(op)
114     if !isLeader {
115         return ErrWrongLeader, ""
116     }
117
118     ch := make(chan applyResult, 1)
119
120     kv.mu.Lock()
121     // Bug1 修复: 若该 index 已有等待项 (极少情况), 通知旧的 WrongLeader
122     if old, exists := kv.notifyChans[index]; exists {
123         old.ch <- applyResult{Err: ErrWrongLeader}
124     }
125     kv.notifyChans[index] = notifyEntry{

```

```

126         clientId: op.ClientId,
127         seqId:    op.SeqId,
128         ch:       ch,
129     }
130     kv.mu.Unlock()
131
132     defer func() {
133         kv.mu.Lock()
134         // 只删自己注册的 entry, 防止把别人的也删掉
135         if e, exists := kv.notifyChans[index]; exists &&
136             e.clientId == op.ClientId && e.seqId == op.SeqId {
137             delete(kv.notifyChans, index)
138         }
139         kv.mu.Unlock()
140     }()
141
142     select {
143     case result := <-ch:
144         if result.ClientId != op.ClientId || result.SeqId != op.SeqId {
145             return ErrWrongLeader, ""
146         }
147         return result.Err, result.Value
148     case <-time.After(ApplyTimeout):
149         return ErrTimeout, ""
150     }
151 }
152
153 // ===== applier =====
154
155 func (kv *KVServer) applier() {
156     for !kv.killed() {
157         msg := <-kv.applyCh
158
159         if msg.SnapshotValid {
160             kv.mu.Lock()
161             if kv.rf.CondInstallSnapshot(msg.SnapshotTerm, msg.SnapshotIndex,
162 msg.Snapshot) {
163                 kv.installSnapshot(msg.Snapshot)
164             }
165             kv.mu.Unlock()
166             continue
167         }
168         if !msg.CommandValid {
169             continue
170         }
171     }

```



```

172         op, ok := msg.Command.(Op)
173         if !ok {
174             continue
175         }
176
177         kv.mu.Lock()
178
179         var result applyResult
180         result.ClientId = op.ClientId
181         result.SeqId = op.SeqId
182
183         if last, dup := kv.dupTable[op.ClientId]; dup && last.SeqId >= op.SeqId
184     {
185         result.Err = last.Reply.Err
186         result.Value = last.Reply.Value
187     } else {
188         r := kv.applyOp(op)
189         result.Err = r.Err
190         result.Value = r.Value
191         if op.OpType != "Get" {
192             kv.dupTable[op.ClientId] = lastReply{SeqId: op.SeqId, Reply:
result}
193         }
194     }
195
196     // Bug2 修复: 先取出 channel, 释放锁后再发送, 避免持锁发送可能引起的死锁
197     var notifyCh chan applyResult
198     if e, exists := kv.notifyChans[msg.CommandIndex]; exists &&
199         e.clientId == op.ClientId && e.seqId == op.SeqId {
200         notifyCh = e.ch
201     }
202
203     needSnapshot := kv.maxraftstate != -1 &&
204         kv.persister.RaftStateSize() >= kv.maxraftstate
205
206     if needSnapshot {
207         kv.doSnapshot(msg.CommandIndex)
208     }
209
210     kv.mu.Unlock()
211
212     // 锁外发送, 彻底避免死锁
213     if notifyCh != nil {
214         notifyCh <- result
215     }
216 }

```

```

217
218 func (kv *KVServer) applyOp(op Op) applyResult {
219     switch op.OpType {
220     case "Get":
221         v, ok := kv.kvStore[op.Key]
222         if !ok {
223             return applyResult{Err: ErrNoKey}
224         }
225         return applyResult{Err: OK, Value: v}
226     case "Put":
227         kv.kvStore[op.Key] = op.Value
228         return applyResult{Err: OK}
229     case "Append":
230         kv.kvStore[op.Key] += op.Value
231         return applyResult{Err: OK}
232     }
233     return applyResult{Err: ErrNoKey}
234 }
235
236 // ===== 快照 =====
237
238 func (kv *KVServer) doSnapshot(index int) {
239     w := new(bytes.Buffer)
240     e := labgob.NewEncoder(w)
241     e.Encode(kv.kvStore)
242     e.Encode(kv.dupTable)
243     kv.rf.Snapshot(index, w.Bytes())
244 }
245
246 func (kv *KVServer) installSnapshot(data []byte) {
247     if len(data) == 0 {
248         return
249     }
250     r := bytes.NewBuffer(data)
251     d := labgob.NewDecoder(r)
252     var kvStore map[string]string
253     var dupTable map[int64]lastReply
254     if d.Decode(&kvStore) != nil || d.Decode(&dupTable) != nil {
255         DPrintf("installSnapshot decode error")
256         return
257     }
258     kv.kvStore = kvStore
259     kv.dupTable = dupTable
260 }
261
262 // ===== Kill =====
263

```

```

264 func (kv *KVServer) Kill() {
265     atomic.StoreInt32(&kv.dead, 1)
266     kv.rf.Kill()
267 }
268
269 func (kv *KVServer) killed() bool {
270     return atomic.LoadInt32(&kv.dead) == 1
271 }
272
273 // ===== StartKVServer =====
274
275 func StartKVServer(servers []*labrpc.ClientEnd, me int, persister
    *raft.Persister, maxraftstate int) *KVServer {
276     labgob.Register(Op{})
277
278     kv := &KVServer{
279         me:          me,
280         maxraftstate: maxraftstate,
281         persister:    persister,
282         kvStore:      make(map[string]string),
283         dupTable:     make(map[int64]lastReply),
284         notifyChans:  make(map[int]notifyEntry),
285     }
286
287     kv.applyCh = make(chan raft.ApplyMsg, 64)
288     kv.rf = raft.Make(servers, me, persister, kv.applyCh)
289
290     if snapshot := persister.ReadSnapshot(); len(snapshot) > 0 {
291         kv.installSnapshot(snapshot)
292     }
293
294     go kv.applier()
295
296     return kv
297 }

```

client.go

```

1 package kvraft
2
3 import (
4     "6.824/labrpc"
5     "crypto/rand"
6     "math/big"
7     "sync"
8     "sync/atomic"

```

```

9     "time"
10 )
11
12 type Clerk struct {
13     servers []*labrpc.ClientEnd
14
15     clientId int64
16     seqId    int64
17     leaderId int
18 }
19
20 // ===== clientId 生成器 =====
21 //
22 // 64bit 布局:
23 // [63:32] timestamp(32bit) - Unix 秒
24 // [31:10] random(22bit)    - 每秒重新采样
25 // [ 9: 0] counter(10bit)   - 同秒内 [0,1023] 回绕
26
27 var (
28     clientIdMu      sync.Mutex
29     clientIdLastSec int64
30     clientIdRand     int64
31     clientIdCounter  int64
32 )
33
34 func genClientId() int64 {
35     clientIdMu.Lock()
36     defer clientIdMu.Unlock()
37
38     now := time.Now().Unix()
39
40     if now != clientIdLastSec {
41         clientIdLastSec = now
42         clientIdRand = randBits(22)
43         clientIdCounter = 0
44     } else {
45         clientIdCounter = (clientIdCounter + 1) & 0x3FF
46     }
47
48     return (now << 32) | (clientIdRand << 10) | clientIdCounter
49 }
50
51 func randBits(bits uint) int64 {
52     max := big.NewInt(1)
53     max.Lsh(max, bits)
54     n, _ := rand.Int(rand.Reader, max)
55     return n.Int64()

```

```

56 }
57
58 func MakeClerk(servers []*labrpc.ClientEnd) *Clerk {
59     return &Clerk{
60         servers: servers,
61         clientId: genClientId(),
62         seqId:    0,
63         leaderId: 0,
64     }
65 }
66
67 func (ck *Clerk) nextSeq() int64 {
68     return atomic.AddInt64(&ck.seqId, 1)
69 }
70
71 // ===== Get =====
72
73 func (ck *Clerk) Get(key string) string {
74     seq := ck.nextSeq()
75     args := &GetArgs{
76         Key:      key,
77         ClientId: ck.clientId,
78         SeqId:    seq,
79     }
80
81     for {
82         reply := &GetReply{}
83         ok := ck.servers[ck.leaderId].Call("KVServer.Get", args, reply)
84
85         if ok {
86             switch reply.Err {
87             case OK:
88                 return reply.Value
89             case ErrNoKey:
90                 return ""
91             case ErrWrongLeader:
92                 // 明确告知不是 leader, 换下一台
93                 ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
94             case ErrTimeout:
95                 // Bug3 修复: 超时不代表未 apply, 用相同 SeqId 继续等待同一 server。
96                 // 若该 server 其实不是 leader, 它会返回 ErrWrongLeader, 届时再换。
97             }
98         } else {
99             // RPC 彻底失败 (网络断开), 才轮转
100             ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
101         }
102     }

```

```

103 }
104
105 // ===== PutAppend =====
106
107 func (ck *Clerk) PutAppend(key string, value string, op string) {
108     seq := ck.nextSeq()
109     args := &PutAppendArgs{
110         Key:    key,
111         Value:   value,
112         Op:      op,
113         ClientId: ck.clientId,
114         SeqId:   seq,
115     }
116
117     for {
118         reply := &PutAppendReply{}
119         ok := ck.servers[ck.leaderId].Call("KVServer.PutAppend", args, reply)
120
121         if ok {
122             switch reply.Err {
123             case OK:
124                 return
125             case ErrWrongLeader:
126                 ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
127             case ErrTimeout:
128                 // 同上: 超时用相同 SeqId 重试, dupTable 保证不重复执行
129             }
130         } else {
131             ck.leaderId = (ck.leaderId + 1) % len(ck.servers)
132         }
133     }
134 }
135
136 func (ck *Clerk) Put(key string, value string) {
137     ck.PutAppend(key, value, "Put")
138 }
139
140 func (ck *Clerk) Append(key string, value string) {
141     ck.PutAppend(key, value, "Append")
142 }

```